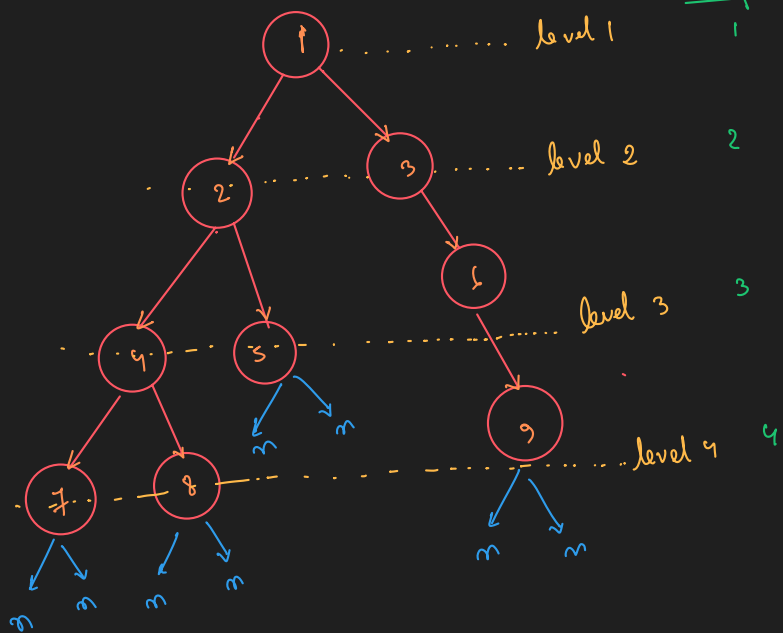
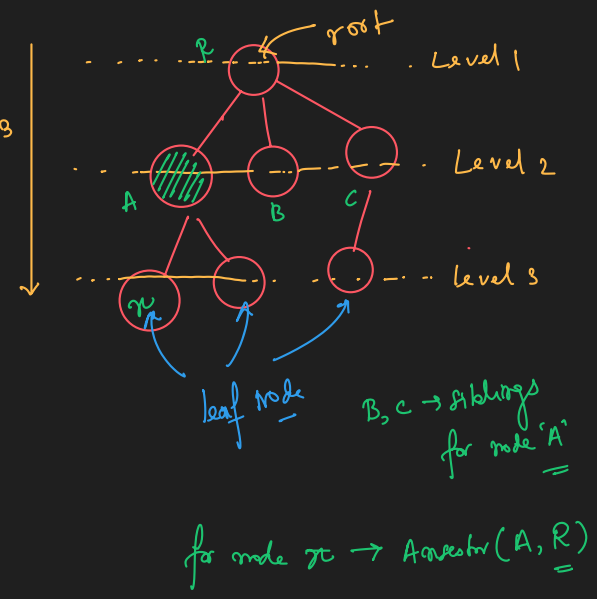


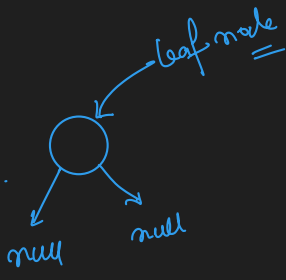
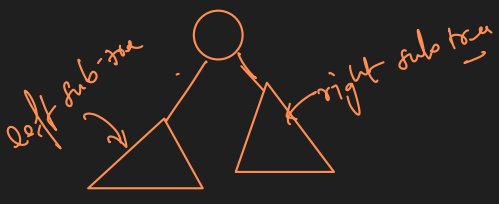
Binary tree



depth 1
depth 2
height = 3



height = 4



class Node {

int data;

Node left, right;

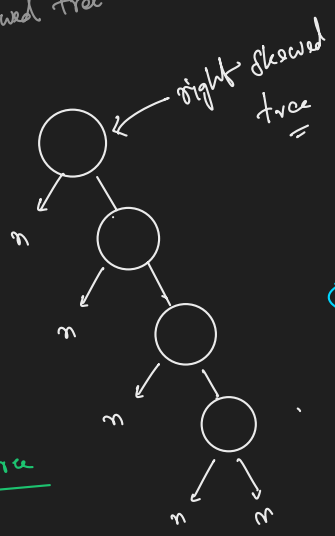
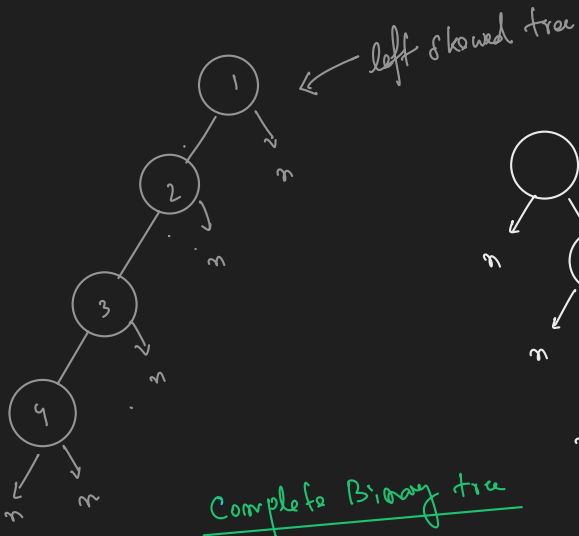
Node (int data) {

this.data = data;

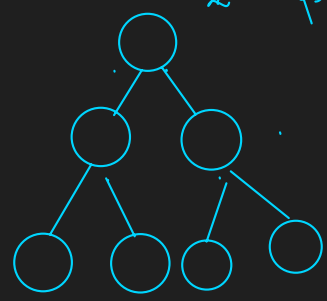
this.left = this.right = null;

}

degree of each node = 2 except the leaf node
perfect Binary tree



l = 3
d = 3



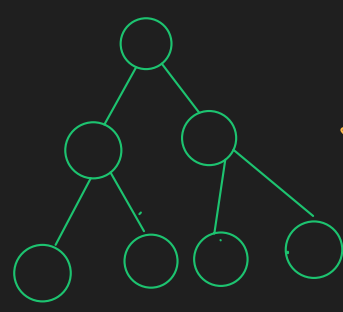
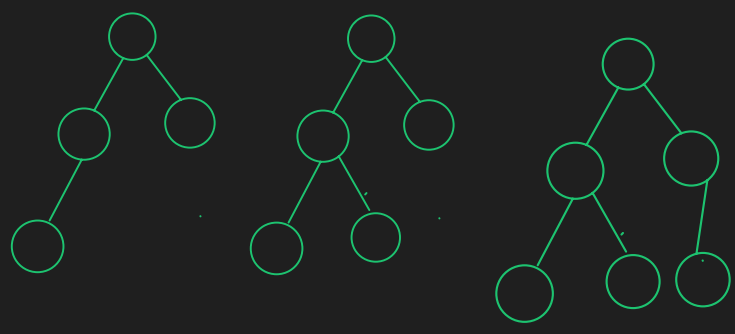
total no of nodes

$$= 2^3 - 1$$

$$= 2^l - 1$$

degree = no of sub nodes

Complete Binary tree



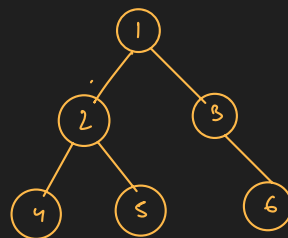
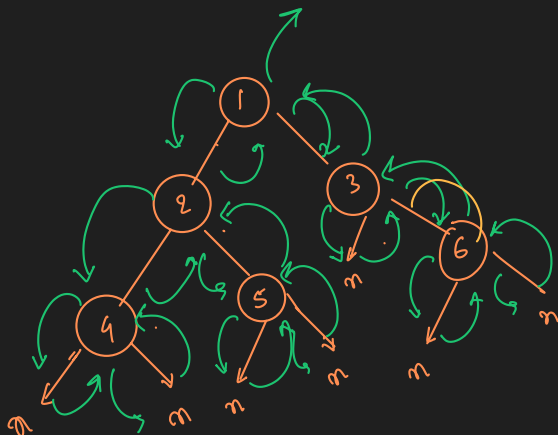
heap

[right subtree will not be there before left subtree/node and fills the node left to right] ← for a complete level

Q Build tree Preorder

if nodes[] = { 1, 2, 4, -1, -1, 5, -1, -1, 3, -1, 6, -1, -1 }

Pre-order Sequence
[root, left, right]



1
2 3
4 5 6

level order



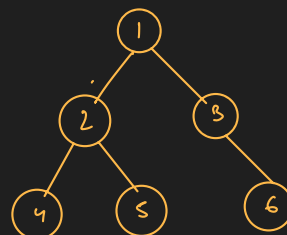
o/p: 1 2 3 4 5 6

```

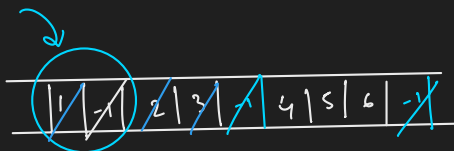
levelOrder (root) {
    if (root == null) {
        q.add (root);
    }
    while (!q.isEmpty()) {
        Node curr = q.remove();
        cout << curr.data;
        if (curr.left != null) {
            q.add (curr.left);
        }
        if (curr.right != null) {
            q.add (curr.right);
        }
    }
}
    
```

```

public static void levelOrder (Node root) {
    Queue<Node> q = new LinkedList<>();
    if (root == null) {
        return;
    }
    q.add (root);
    while (!q.isEmpty()) {
        Node curr = q.remove();
        System.out.print (curr.data + " ");
        if (curr.left != null) {
            q.add (curr.left);
        }
        if (curr.right != null) {
            q.add (curr.right);
        }
    }
}
    
```



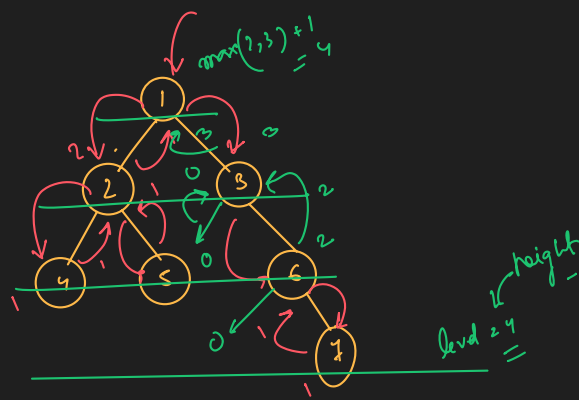
initial standing



o/p: 1
2 3
4 5 6

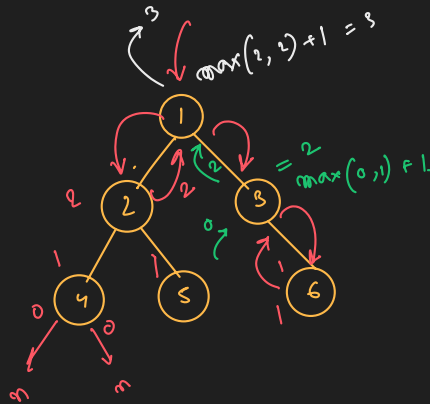
height of a tree

Time Complexity?



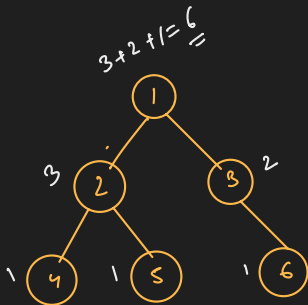
$$\text{height} = \max(\text{lh}, \text{rh}) + 1$$

null(node)
↳ height = 0



```
public static int height(Node root){
    if(root == null){
        return 0;
    }
    return Math.max(height(root.left), height(root.right)) + 1;
}
```

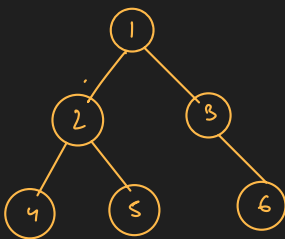
Count of the nodes



$$\text{count} = \text{left count} + \text{right count} + 1 \quad \leftarrow \text{itself}$$

```
public static int count(Node root){
    if(root == null){
        return 0;
    }
    return count(root.left) + count(root.right) + 1;
}
```

Sum of the nodes



$$\text{Sum} = \text{left sum} + \text{right sum} + \text{root.data}$$

```
public static int sum(Node root){
    if(root == null){
        return 0;
    }
    return sum(root.left) + sum(root.right) + root.data;
}
```